

Q1)

1. Design:

```
1  import testing_pkg::*;
2
3  module alu_seq(operand1, operand2, clk, rst, opcode, out);
4  input byte operand1, operand2;
5  input clk, rst;
6  input opcode_e opcode;
7  output byte out;
8
9  always @(posedge clk) begin
10     if (rst)
11         out <= 0;
12     else
13         case (opcode)
14             ADD: out <= operand1 + operand2;
15             SUB: out <= operand1 - operand2;
16             MULT: out <= operand1 * operand2;
17             DIV: out <= operand1 / operand2;
18             default: out <= 0;
19         endcase
20     end
21 endmodule
```

2. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALU_1	Asserts the reset signal for one cycle then drops it to zero and reasserts it again then to zero for full toggle coverage	At the beginning	-	A checker to make sure output is correct
ALU_2	Directed tests for operand1 and opcode conditions to achieve full code and functional coverage	Directed after ALU_1 during simulation	Covers the values of opcode_operand1	A checker to make sure output is correct
ALU_3	Random tests to achieve the remaining code and functional coverages	Directed after ALU_1 during simulation	Covers the values of opcode_cp and operand1_cp	A checker to make sure output is correct

3. Package Code:

```
1 package testing_pkg;
2
3     typedef enum {ADD, SUB, MULT, DIV} opcode_e;
4
5     class Transaction;
6
7         rand opcode_e opcode;
8         rand byte operand1;
9         rand byte operand2;
10        bit clk;
11
12        //To allow more chances for Crossing to happen
13        constraint c_special_vals {
14            operand1 dist { -128 := 10, 127 := 10, 0 := 10, [-127:126] := 1 };
15        }
16
17        constraint c_opcode {
18            opcode dist {[ADD:SUB]:/80, [MULT: DIV]:/20 };
19        }
20
21        covergroup CovCode @(posedge clk);
22
23            operand1_cp: coverpoint operand1{
24                bins zero = {0};
25                bins MAXNEG = {-128};
26                bins MAXPOS = {127};
27                bins misc = default;
28            }
29
30            opcode_cp: coverpoint opcode{
31                bins add_sub = {ADD, SUB};
32                bins add_to_sub = (ADD => SUB);
33                illegal_bins no_DIV = {DIV};
34                bins misc = default;
35            }
36
37            opcode_operand1: cross opcode_cp, operand1_cp {
38                option.weight = 5;
39                option.cross_auto_bin_max = 0;
40                bins ADD_SUB_MAXPOS = binsof(opcode_cp.add_sub) && binsof(operand1_cp.MAXPOS);
41                bins ADD_SUB_MAXNEG = binsof(opcode_cp.add_sub) && binsof(operand1_cp.MAXNEG);
42            }
43
44        endgroup
45
46        function new();
47            CovCode = new;
48        endfunction
49
50    endclass
51 endpackage
```

4. Testbench Code:

```
1  import testing_pkg::*;
2
3  module tb ();
4
5      Transaction obj = new;
6
7      byte operand1, operand2;
8      logic clk, rst;
9      opcode_e opcode;
10     byte out;
11
12     int error_count, correct_count;
13
14     alu_seq DUT (.*);
15
16     initial begin
17         clk = 0;
18         forever begin
19             #1 clk = ~clk;
20             obj.clk = clk;
21         end
22     end
23
24     initial begin
25
26         error_count = 0; correct_count = 0;
27
28         reset_assert();
29
30         repeat(50)begin //Increased the number of tests from 32 to 50
31             assert(obj.randomize());
32             operand1 = obj.operand1;
33             operand2 = obj.operand2;
34             opcode = obj.opcode;
35             check_result();
36
37         end
38         $display("Total Tests:%0d | Correct Tests:%0d | Error Test:%0d", correct_count + error_count, correct_count, error_count);
39         $stop;
40     end
41 end
```

```

41
42     task reset_assert();
43         rst = 1;
44         check_result();
45         rst = ~rst;
46     endtask
47
48     task clac_result(output byte out_expected);
49
50         if (rst)
51             out_expected = 0;
52         else
53             case (opcode)
54                 ADD: out_expected = operand1 + operand2;
55                 SUB: out_expected = operand1 - operand2;
56                 MULT: out_expected = operand1 * operand2;
57                 DIV: out_expected = operand1 / operand2;
58                 default: out_expected = 0;
59             endcase
60     endtask
61
62     task check_result();
63
64         byte out_expected;
65         clac_result(out_expected);
66         @(negedge clk);
67         if(out != out_expected) begin
68             error_count++;
69             $display("Expected:%d | DUT:%d <---Error", out_expected, out);
70         end
71         else begin
72             $display("Expected:%d | DUT:%d", out_expected, out);
73             correct_count++;
74         end
75     endtask
76
77
78
79     endmodule

```

5. Do File

```
1  vlib work
2  vlog testing_pkg.sv alu.sv tb.sv +cover -covercells
3  vsim -voptargs=+acc work.tb -cover
4  do wave.do
5  coverage save alu.ucdb -onexit -du work.alu_seq
6  run -all
7  quit -sim
8  vcover report ALSU_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

6. Coverage Report:

Branch Coverage:

```
=====
=== Instance: /\tb#DUT
=== Design Unit: work.alu_seq
=====
Branch Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Branches                  6        6         0    100.00%

=====Branch Details=====
```

Statement Coverage:

```
Statement Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Statements                 6        6         0    100.00%

=====Statement Details=====
```

Toggle & Total Coverage:

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	-----
Toggles	56	56	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\tb#DUT --

Node	1H->0L	0L->1H			"Coverage"
clk	1	1			100.00
opcode		ENUM type	Value	Count	
		ADD	6	100.00	
		SUB	6	100.00	
		MULT	1	100.00	
		DIV	2	100.00	
operand1[7-0]	1	1			100.00
operand2[7-0]	1	1			100.00
out[7-0]	1	1			100.00
rst	1	1			100.00

Total Node Count = 30

Toggled Node Count = 30

Untoggled Node Count = 0

Toggle Coverage = 100.00% (56 of 56 bins)

Total Coverage By Instance (filtered view): 100.00%

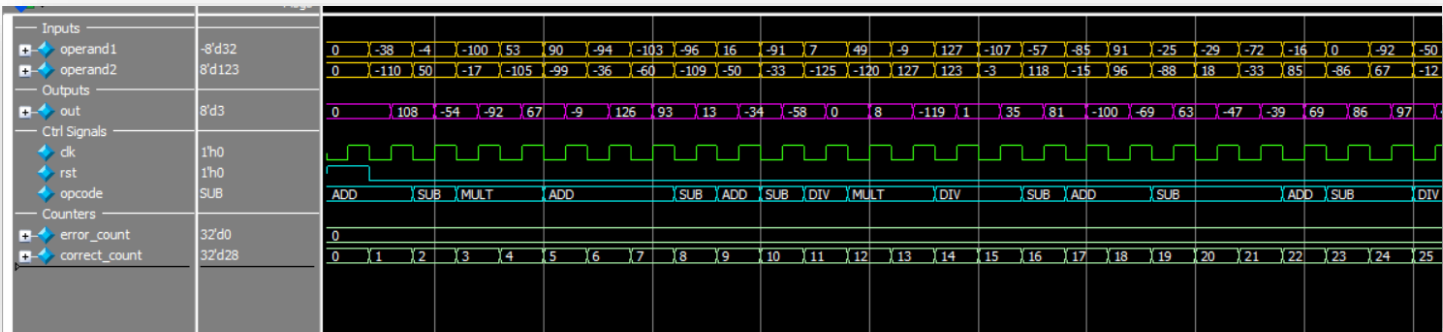
7. Functional Coverage Report:

```
Covergroup instance /\testing_pkg::Transaction::CovCode
100.00%      100      - Covered
covered/total bins:      7      7      -
missing/total bins:      0      7      -
% Hit:      100.00%      100      -
Coverpoint operand1_cp      100.00%      100      - Covered
covered/total bins:      3      3      -
missing/total bins:      0      3      -
% Hit:      100.00%      100      -
bin zero      3      1      - Covered
bin MAXNEG      2      1      - Covered
bin MAXPOS      5      1      - Covered
default bin misc      72      - Occurred
Coverpoint opcode_cp      100.00%      100      - Covered
covered/total bins:      2      2      -
missing/total bins:      0      2      -
% Hit:      100.00%      100      -
illegal_bin no_DIV      8      - Occurred
bin add_sub      63      1      - Covered
bin add_to_sub      12      1      - Covered
default bin misc      11      - Occurred
Cross opcode_operand1      100.00%      100      - Covered
covered/total bins:      2      2      -
missing/total bins:      0      2      -
% Hit:      100.00%      100      -
Auto, Default and User Defined Bins:
bin ADD_SUB_MAXPOS      5      1      - Covered
bin ADD_SUB_MAXNEG      2      1      - Covered

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

Total Coverage By Instance (filtered view): 100.00%
```

8. Questasim Waveform:



Q2)

1. Design

```
1  //////////////////////////////////////
2  // Author: Kareem Waseem
3  // Course: Digital Verification using SV & UVM
4  //
5  // Description: Counter Design
6  //
7  //////////////////////////////////////
8  module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9  parameter WIDTH = 4;          //(Valid values: 4, 6, 8, default: 4)
10 input clk;
11 input rst_n;                  //(active low sync rst)
12 input load_n;                 //(active low load)
13 input up_down;                //high then increment counter, else decrement
14 input ce;                     //(clock enable signal
15 input [WIDTH-1:0] data_load;  //(load data to count_out output when the load_n signal is asserted)
16 output reg [WIDTH-1:0] count_out;
17 output max_count;             //counter reaches the maximum value, this signal is high, else low)
18 output zero;                  //counter reaches the minimum value, this signal is high, else low)
19
20 always @(posedge clk) begin
21     if (~rst_n)                //Modification: changed comparison "!" to bitwise NOT "~"
22         count_out <= 0;
23     else if (~load_n)           //Modification: changed comparison "!" to bitwise NOT "~"
24         count_out <= data_load;
25     else if (ce)begin          //Fixed BUG: added Begin and End statments
26         if (up_down)
27             count_out <= count_out + 1;
28         else
29             count_out <= count_out - 1;
30     end
31 end
32
33 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
34 assign zero = (count_out == 0)? 1:0;
35
36 endmodule
```

2. Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When reset is asserted the output of the counter should be low	Directed at the beginning of the sim then randomized with the constraint that drives the reset to be off most of the simulation time	-	A checker to make sure output is correct
COUNTER_2	When load_n is asserted the output should take the value of the data_load input	Randomization under the load signal to be off %70 of the time	Cover all values of data_load	A checker to make sure output is correct
COUNTER_3	When ce is asserted the counter should count up or down based on the up_down signal	Randomization under the ce signal to be on %70 of the time	Cover all values of count_up and transition bin from max to zero	A checker to make sure output is correct

Package Code

```
1 package counter_pkg;
2
3 parameter WIDTH = 4;
4 localparam MAX_VAL = (1 << WIDTH) - 1;
5
6 class Counter_inputs;
7
8     rand logic rst_n;                //(active low sync rst)
9     rand logic load_n;               //(active low load)
10    rand logic up_down;               //high then increment counter, else decrement
11    rand logic ce;                    //(clock enable signal
12    rand logic [WIDTH-1:0] data_load; //(load data to count_out output when the load_n signal is asserted)
13
14    logic [WIDTH-1:0] count_out;
15    bit clk;
16
17    constraint c_rst_n{
18        rst_n dist{0:/0, 1:/100};
19    }
20
21    constraint c_load_n{
22        load_n dist{0:/30, 1:/70};
23    }
24
25    constraint c_ce{
26        ce dist{0:/30, 1:/70};
27    }
28
29    covergroup CovCode @(posedge clk);
30
31        load_data_cp: coverpoint data_load iff(rst_n && ~load_n);
32
33        count_out_updown_high_cp : coverpoint count_out iff(rst_n && ce && up_down);
34
35        count_out_trans_updown_high_cp : coverpoint count_out iff(rst_n && ce && up_down){
36            bins OV_MAX_ZERO = (4'hf => 0);
37        }
38
39        count_out_updown_low_cp : coverpoint count_out iff(rst_n && ce && ~up_down);
40
41        count_out_trans_updown_low_cp : coverpoint count_out iff(rst_n && ce && ~up_down){
42            bins OV_ZERO_MAX = (0 => 4'hf);
43        }
44
45    endgroup
46
47
48    function new(logic rst_n = 1, logic load_n = 1, logic up_down = 1, logic ce = 0,
49        logic [WIDTH-1:0] data_load = 0);
50
51        this.rst_n = rst_n;
52        this.load_n = load_n;
53        this.up_down = up_down;
54        this.ce = ce;
55        this.data_load = data_load;
56
57        CovCode = new;
58
59    endfunction
60
61
62 endclass
63
64 endpackage
```

3. Testbench Code:

```
1  import counter_pkg::*;
2
3  module counter_tb();
4
5      logic clk;
6      logic rst_n;                //(active low sync rst)
7      logic load_n;               //(active low load)
8      logic up_down;              //high then increment counter, else decrement
9      logic ce;                   //(clock enable signal
10     logic [WIDTH-1:0] data_load; //(load data to count_out output when the load_n signal is asserted)
11     logic [WIDTH-1:0] count_out;
12     logic [WIDTH-1:0] count_out_expected;
13     logic max_count;             //counter reaches the maximum value, this signal is high, else low)
14     logic zero;                  //counter reaches the minimum value, this signal is high, else low)
15
16     counter #(WIDTH) DUT (.*)
17     N_bit_up_down_Counter #(WIDTH) Golden(.clk(clk),
18     .reset(rst_n),
19     .en_load(load_n),
20     .clk_en(ce),
21     .load(data_load),
22     .up_ndown(up_down),
23     .cnt(count_out_expected));
24
25     Counter_inputs inputs = new();
26
27     int error_count, correct_count;
28
29     initial begin
30         clk = 0;
31         forever begin
32             #1 clk = ~clk;
33             inputs.clk = clk;
34             inputs.count_out = count_out;
35         end
36     end
37
38     task check_result();
39     @(negedge clk);
40     $display("Expected:%d | DUT:%d | UP_DOWN:%d | CLK_EN:%d | Load_n:%d | Load Data:%d", count_out_expected, count_out
41     , up_down, ce, load_n, data_load);
42
43     if(count_out != count_out_expected) begin
44         error_count++;
45         // $stop;
46     end
47     else begin
48         correct_count++;
49     end
50 endtask
51
52 initial begin
53     assert(inputs.randomize());
54     //COUNTER_1
55     rst_n = 0;
56     load_n = inputs.load_n;
57     up_down = inputs.up_down;
58     ce = inputs.ce;
59     data_load = inputs.data_load;
60
61     check_result();
62
63     //COUNTER_2, 3, 4, 5
64     for(int i = 0; i < 100; i++)begin
65         assert(inputs.randomize());
66         rst_n = inputs.rst_n;
67         load_n = inputs.load_n;
68         up_down = 1; inputs.up_down = 1;
69         ce = inputs.ce;
70         data_load = inputs.data_load;
71
72         check_result();
73     end
74 end
```



```
1
2     for(int i = 0; i < 100; i++)begin
3         assert(inputs.randomize());
4         rst_n = inputs.rst_n;
5         load_n = inputs.load_n;
6         up_down = 0; inputs.up_down = 0;
7         ce = inputs.ce;
8         data_load = inputs.data_load;
9
10        check_result();
11    end
12
13    //COUNTER_6
14    rst_n = 1; load_n = 0; ce = 1; data_load = 4'b1111;
15    @(negedge clk)
16    if(max_count != 1)begin
17        $display("Error In Max_Count: Expected:1 | DUT:
18", max_count); error_count++;
19        // $stop;
20    end
21    else begin
22        correct_count++;
23    end
24
25    //COUNTER_7
26    rst_n = 1; load_n = 0; ce = 1; data_load = 0;
27    @(negedge clk)
28    if(zero != 1)begin
29        $display("Error In zero: Expected:1 | DUT:
30", zero); error_count++;
31        // $stop;
32    end
33    else begin
34        correct_count++;
35    end
36
37    //Additional Tests for 100% Coverage
38    assert(inputs.randomize());
39    rst_n = 0;
40    load_n = inputs.load_n;
41    up_down = inputs.up_down;
42    ce = inputs.ce;
43    data_load = inputs.data_load;
44
45    check_result();
46
47    assert(inputs.randomize());
48    rst_n = inputs.rst_n;
49    load_n = inputs.load_n;
50    up_down = 1; inputs.up_down = 1;
51    ce = inputs.ce;
52    data_load = inputs.data_load;
53
54    check_result();
55
56
57    $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
58    $stop;
59
60    end
61
62 endmodule
```

5. Do File

```
1 vlib work
2 vlog counter_pkg.sv counter.v counter_golden.v counter_tb.sv +cover -covercells
3 vsim -voptargs=+acc work.counter_tb -cover
4 do wave.do
5 coverage save counter_tb.ucdb -onexit -du work.counter
6 run -all
7 quit -sim
8 vcover report counter_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

6. Coverage Report

Branch Coverage:

```
=====
=== Instance: /\counter_tb#DUT
=== Design Unit: work.counter
=====
Branch Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Branches                  10       10         0    100.00%
=====Branch Details=====
```

Statement Coverage:

```
Statement Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Statements                 7         7         0    100.00%
=====Statement Details=====
```

Toggle & Total Coverage:

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\counter_tb#DUT --

	Node	1H->0L	0L->1H	"Coverage"
	-----	-----	-----	-----
	ce	1	1	100.00
	clk	1	1	100.00
	count_out[3-0]	1	1	100.00
	data_load[0-3]	1	1	100.00
	load_n	1	1	100.00
	max_count	1	1	100.00
	rst_n	1	1	100.00
	up_down	1	1	100.00
	zero	1	1	100.00

Total Node Count = 15
 Toggled Node Count = 15
 Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

Total Coverage By Instance (filtered view): 100.00%

7. Functional Coverage:

TYPE /\counter_pkg/Counter_inputs/CovCode	100.00%	100	-	Covered
covered/total bins:	50	50	-	
missing/total bins:	0	50	-	
% Hit:	100.00%	100	-	
Coverpoint load_data_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_out_updown_high_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_out_trans_updown_high_cp	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Coverpoint count_out_updown_low_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_out_trans_updown_low_cp	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Covergroup instance /\counter_pkg::Counter_inputs::CovCode	100.00%	100	-	Covered
covered/total bins:	50	50	-	
missing/total bins:	0	50	-	
% Hit:	100.00%	100	-	

Coverpoint load_data_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	9	1	-	Covered
bin auto[1]	2	1	-	Covered
bin auto[2]	4	1	-	Covered
bin auto[3]	6	1	-	Covered
bin auto[4]	4	1	-	Covered
bin auto[5]	4	1	-	Covered
bin auto[6]	8	1	-	Covered
bin auto[7]	3	1	-	Covered
bin auto[8]	6	1	-	Covered
bin auto[9]	6	1	-	Covered
bin auto[10]	3	1	-	Covered
bin auto[11]	1	1	-	Covered
bin auto[12]	5	1	-	Covered
bin auto[13]	2	1	-	Covered
bin auto[14]	4	1	-	Covered
bin auto[15]	2	1	-	Covered
Coverpoint count_out_updown_high_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	8	1	-	Covered
bin auto[1]	5	1	-	Covered
bin auto[2]	4	1	-	Covered
bin auto[3]	7	1	-	Covered
bin auto[4]	5	1	-	Covered
bin auto[5]	4	1	-	Covered
bin auto[6]	7	1	-	Covered
bin auto[7]	4	1	-	Covered
bin auto[8]	5	1	-	Covered
bin auto[9]	6	1	-	Covered
bin auto[10]	5	1	-	Covered
bin auto[11]	1	1	-	Covered
bin auto[12]	2	1	-	Covered
bin auto[13]	2	1	-	Covered
bin auto[14]	2	1	-	Covered
bin auto[15]	3	1	-	Covered
Coverpoint count_out_trans_updown_high_cp	100.00%	100	-	Covered

```

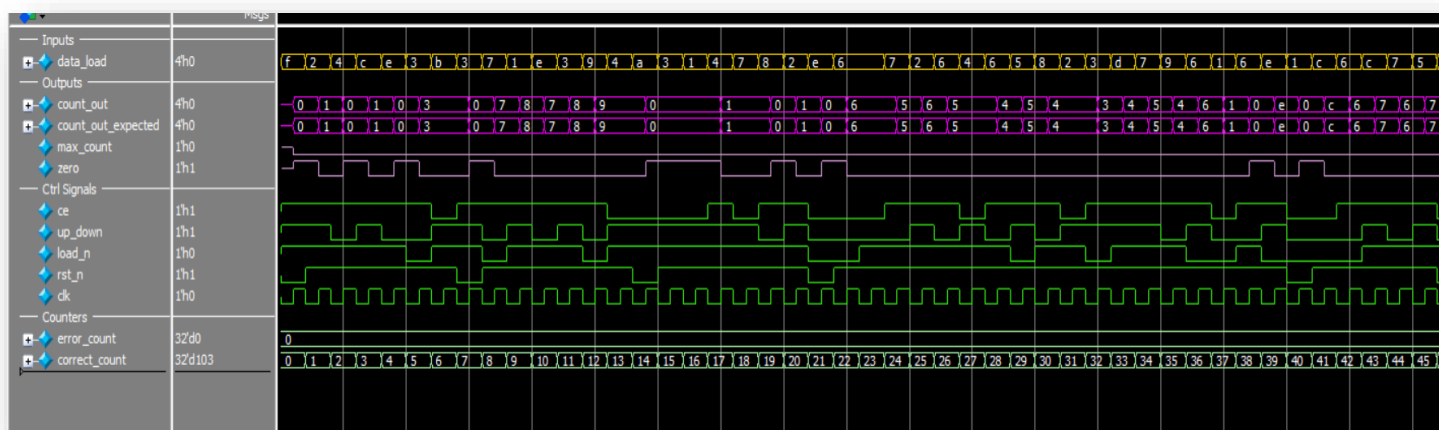
Coverpoint count_out_trans_updown_high_cp      100.00%      100      -      Covered
covered/total bins:                          1          1      -
missing/total bins:                          0          1      -
% Hit:                                        100.00%      100      -
bin OV_MAX_ZERO                               2          1      -      Covered
Coverpoint count_out_updown_low_cp              100.00%      100      -      Covered
covered/total bins:                          16         16      -
missing/total bins:                          0          16      -
% Hit:                                        100.00%      100      -
bin auto[0]                                   6          1      -      Covered
bin auto[1]                                   5          1      -      Covered
bin auto[2]                                   7          1      -      Covered
bin auto[3]                                   4          1      -      Covered
bin auto[4]                                   3          1      -      Covered
bin auto[5]                                   4          1      -      Covered
bin auto[6]                                   9          1      -      Covered
bin auto[7]                                   6          1      -      Covered
bin auto[8]                                   6          1      -      Covered
bin auto[9]                                   3          1      -      Covered
bin auto[10]                                  3          1      -      Covered
bin auto[11]                                  2          1      -      Covered
bin auto[12]                                  3          1      -      Covered
bin auto[13]                                  2          1      -      Covered
bin auto[14]                                  3          1      -      Covered
bin auto[15]                                  5          1      -      Covered
Coverpoint count_out_trans_updown_low_cp        100.00%      100      -      Covered
covered/total bins:                          1          1      -
missing/total bins:                          0          1      -
% Hit:                                        100.00%      100      -
bin OV_ZERO_MAX                               4          1      -      Covered

TOTAL COVERGROUP COVERAGE: 100.00%  COVERGROUP TYPES: 1

Total Coverage By Instance (filtered view): 100.00%

```

8. Questasim Waveform:



Q3)

1. Design

```
1  module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2  parameter INPUT_PRIORITY = "A";
3  parameter FULL_ADDER = "ON";
4  input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5  input [2:0] opcode;
6  input signed [2:0] A, B;
7  output reg [15:0] leds;
8  output reg signed [5:0] out;
9
10 reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11 reg cin_reg; //Fixed Bug: CIN supposed to be 1 bit unsigned not 2 bits signed
12 reg [2:0] opcode_reg;
13 reg signed [2:0] A_reg, B_reg;
14
15 wire invalid_red_op, invalid_opcode, invalid;
16
17 //Invalid handling
18 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
19 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20 assign invalid = invalid_red_op | invalid_opcode;
21
22 //Registering input signals
23 always @(posedge clk or posedge rst) begin
24     if(rst) begin
25         cin_reg <= 0;
26         red_op_B_reg <= 0;
27         red_op_A_reg <= 0;
28         bypass_B_reg <= 0;
29         bypass_A_reg <= 0;
30         direction_reg <= 0;
31         serial_in_reg <= 0;
32         opcode_reg <= 0;
33         A_reg <= 0;
34         B_reg <= 0;
35     end else begin
36         cin_reg <= cin;
37         red_op_B_reg <= red_op_B;
38         red_op_A_reg <= red_op_A;
39         bypass_B_reg <= bypass_B;
40         bypass_A_reg <= bypass_A;
41         direction_reg <= direction;
42         serial_in_reg <= serial_in;
43         opcode_reg <= opcode;
44         A_reg <= A;
45         B_reg <= B;
46     end
47 end
48
49 //leds output blinking
50 always @(posedge clk or posedge rst) begin
51     if(rst) begin
52         leds <= 0;
53     end else begin
54         if (invalid)
55             leds <= ~leds;
56         else
57             leds <= 0;
58     end
59 end
60
```

```

60
61 //ALSU output processing
62 always @(posedge clk or posedge rst) begin
63     if(rst) begin
64         out <= 0;
65     end
66     else begin
67         if (bypass_A_reg && bypass_B_reg)
68             out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69         else if (bypass_A_reg)
70             out <= A_reg;
71         else if (bypass_B_reg)
72             out <= B_reg;
73         else if (invalid)
74             out <= 0;
75         else begin
76             case (opcode_reg) //Fixed Bug: changed incorrect checking opcode to opcode_reg
77                 3'h0: begin
78                     if (red_op_A_reg && red_op_B_reg)
79                         out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
80                     else if (red_op_A_reg)
81                         out <= |A_reg;
82                     else if (red_op_B_reg)
83                         out <= |B_reg;
84                     else
85                         out <= A_reg | B_reg;
86                 end
87                 3'h1: begin
88                     if (red_op_A_reg && red_op_B_reg)
89                         out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90                     else if (red_op_A_reg)
91                         out <= ^A_reg;
92                     else if (red_op_B_reg)
93                         out <= ^B_reg;
94                     else
95                         out <= A_reg ^ B_reg;
96                 end
97                 3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
98                 3'h3: out <= A_reg * B_reg;
99                 3'h4: begin
100                     if (direction_reg)
101                         out <= {out[4:0], serial_in_reg};
102                     else
103                         out <= {serial_in_reg, out[5:1]};
104                 end
105                 3'h5: begin
106                     if (direction_reg)
107                         out <= {out[4:0], out[5]};
108                     else
109                         out <= {out[0], out[5:1]};
110                 end
111                 default: out <= 0;
112             endcase
113         end
114     end
115 end
116
117 endmodule

```

2. Do file:

```

1  vlib work
2  vlog ALSU.v ALSU_Golden_2.v D_FF.v ALSU_tb.sv ALSU_pkg.sv +cover -covercells
3  vsim -voptargs=+acc work.ALSU_tb -cover
4  do wave.do
5  coverage save ALSU_tb.ucdb -onexit -du work.ALSU
6  run -all
7  coverage exclude -src ALSU.v -line 111 -code s
8  coverage exclude -src ALSU.v -line 111 -code b
9  quit -sim
10 vcover report ALSU_tb.ucdb -details -annotate -all -output coverage_rpt.txt

```

3. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	When reset is asserted , output should be low	Directed at the beginning then randomized with the constraint that drives the reset to be off most of the time	-	A checker to make sure output is correct
ALSU_2	When red_op_A and red_op_B are both asserted , and the opcode is OR_OP , tests for output prioritizing A reduction	Directed after ALSU_1 , during simulation	-	A checker to make sure output is correct
ALSU_3	When red_op_A and red_op_B are both asserted , and the opcode is XOR_OP , tests for output prioritizing A reduction	Directed after ALSU_2 , during simulation	-	A checker to make sure output is correct
ALSU_4	Randomizes opcode values , and tests outputs for various inputs while applying different constraints	Directed in Randomization part during simulation	Covers A_cp ,B_cp , ALU_cp	A checker to make sure output is correct
ALSU_5	Randomizes opcode values , and tests outputs for various opcodes while without most constraints	Directed in Randomization part during simulation	Covers A_cp ,B_cp , ALU_cp	A checker to make sure output is correct

4. Test Bench Code:

Package(added Covergroup):

```
62
63     covergroup CovCode @(posedge clk);
64
65     A_CP: coverpoint A {
66         bins A_data_0 = {0};
67         bins A_data_max = {MAXPOS};
68         bins A_data_min = {MAXNEG};
69         bins A_data_default = default;
70         bins A_data_walkingones[] = {3'b001, 3'b010, 3'b100} iff(red_op_A);
71     }
72
73     B_CP: coverpoint B {
74         bins B_data_0 = {0};
75         bins B_data_max = {MAXPOS};
76         bins B_data_min = {MAXNEG};
77         bins B_data_default = default;
78         bins B_data_walkingones[] = {3'b001, 3'b010, 3'b100} iff(red_op_B && ~red_op_A);
79     }
80
81     ALU_cp: coverpoint opcode {
82         bins Bins_shift[] = {SHIFT, ROTATE};
83         bins Bins_arith[] = {ADD, MULT};
84         bins Bins_bitwise[] = {OR, XOR};
85         bins Bins_trans = (0 => 1 => 2 => 3 => 4 => 5);
86         illegal_bins Bins_invalid = {INVALID_6, INVALID_7};
87     }
88
89     endgroup
90
91     function new(cin = 0, rst = 0, red_op_A = 0, red_op_B = 0, bypass_A = 0,
92         bypass_B = 0, direction = 0, serial_in = 0,
93         A = 0, B = 0, OPCODE_e opcode = OR);
94         this.cin = cin;
95         this.rst = rst;
96         this.red_op_A = red_op_A;
97         this.red_op_B = red_op_B;
98         this.bypass_A = bypass_A;
99         this.bypass_B = bypass_B;
100         this.direction = direction;
101         this.serial_in = serial_in;
102         this.opcode = opcode;
103         this.A = A;
104         this.B = B;
105
106         CovCode = new;
107
108     endfunction //new()
109
110 endclass //ALSU_inputs
111
112 endpackage
```

Test Bench Code:

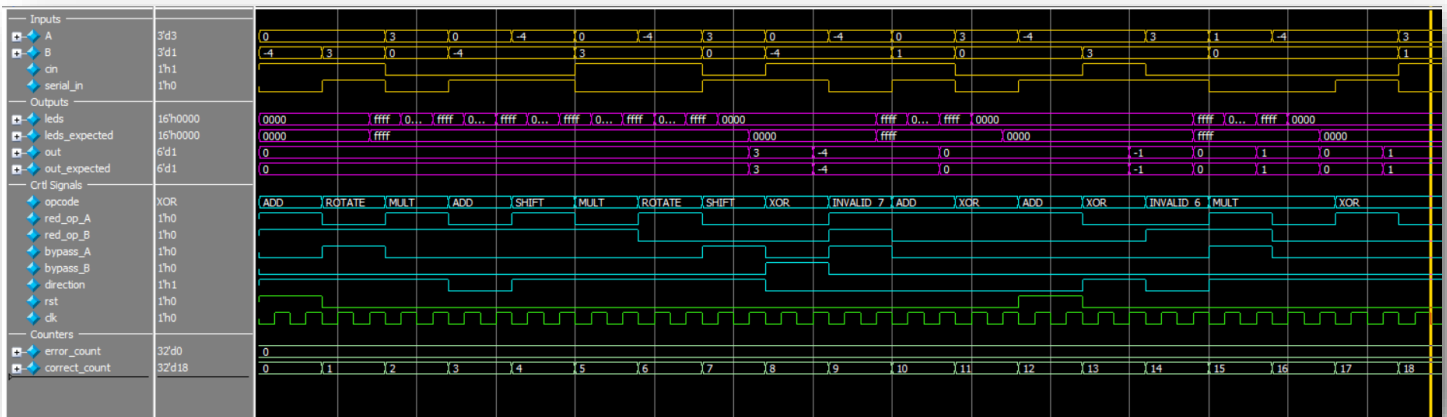
```
1 import ALSU_pkg::*;
2
3 module ALSU_tb ();
4
5     logic clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
6     OPCODE_e opcode;
7     logic signed [2:0] A, B;
8     logic [15:0] leds;
9     logic signed [5:0] out;
10    //Testbench Variables
11    logic [15:0] leds_expected;
12    logic signed [5:0] out_expected;
13
14    ALSU_inputs inputs = new();
15    int error_count, correct_count;
16
17    ALSU DUT (.*);
18    ALSU_golden_2 Golden (.clk(clk), .A(A), .B(B), .cin(cin), .rst(rst), .red_op_A(red_op_A),
19        .red_op_B(red_op_B), .bypass_A(bypass_A), .bypass_B(bypass_B),
20        .direction(direction), .serial_in(serial_in), .opcode(opcode),
21        .leds(leds_expected), .out_Off(out_expected));
22
23    initial begin
24        clk = 0;
25        forever
26            #1 clk = ~clk;
27    end
28
29    initial begin
30        //ALSU_1
31        assert(inputs.randomize());
32        rst = 1;
33        cin = inputs.cin;
34        red_op_A = inputs.red_op_A;
35        red_op_B = inputs.red_op_B;
36        bypass_A = inputs.bypass_A;
37        bypass_B = inputs.bypass_B;
38        direction = inputs.direction;
39        serial_in = inputs.serial_in;
40        opcode = inputs.opcode;
41
42        A = inputs.A;
43        B = inputs.B;
44        check_result();
45
46        //ALSU_2, ALSU_3, ALSU_5
47        for(int i = 0; i < 200; i++)begin
48            assert(inputs.randomize());
49            rst = inputs.rst;
50            cin = inputs.cin;
51            red_op_A = inputs.red_op_A;
52            red_op_B = inputs.red_op_B;
53            bypass_A = inputs.bypass_A;
54            bypass_B = inputs.bypass_B;
55            direction = inputs.direction;
56            serial_in = inputs.serial_in;
57            opcode = inputs.opcode;
58
59            A = inputs.A;
60            B = inputs.B;
61            check_result();
62        end
63
64        //ALSU_4
65        A = 2; B = 3;
66        cin = 0; serial_in = 0;
67        red_op_A = 1; red_op_B = 1;
68        opcode = OR; direction = 1;
69        bypass_A = 0; bypass_B = 0;
70        check_result();
71
72        A = 2; B = 3;
73        cin = 0; serial_in = 0;
74        red_op_A = 1; red_op_B = 0;
75        opcode = OR; direction = 1;
76        bypass_A = 0; bypass_B = 0;
77        check_result();
78
79        A = 2; B = 3;
80        cin = 0; serial_in = 0;
81        red_op_A = 0; red_op_B = 1;
82        opcode = OR; direction = 1;
83        bypass_A = 0; bypass_B = 0;
84        check_result();
85
86        A = -1; B = 1;
87        cin = 0; serial_in = 0;
88        red_op_A = 1; red_op_B = 1;
89        opcode = XOR; direction = 1;
90        bypass_A = 0; bypass_B = 0;
91        check_result();
92    end
```

```

1
2 //Additional Tests for 100% functional Coverage
3 inputs.opcode = OR; opcode = OR;
4 for(int i = 0; i <= 6; i++)begin
5     assert(inputs.randomize());
6     rst          = 0;
7     inputs.rst    = 0;
8     cin          = inputs.cin;
9     red_op_A      = 0;
10    red_op_B       = 0;
11    inputs.red_op_A = 0;
12    inputs.red_op_B = 0;
13    bypass_A       = inputs.bypass_A;
14    bypass_B       = inputs.bypass_B;
15    direction      = inputs.direction;
16    serial_in      = inputs.serial_in;
17
18    A = inputs.A;
19    B = inputs.B;
20    check_result();
21    opcode = opcode.next;
22    inputs.opcode = opcode.next;
23 end
24
25
26 $display("Total Tests:
27 | Correct:
28 | Error:
29 ", correct_count + error_count, correct_count, error_count);
30 task check_result();
31     repeat(2)@(negedge clk);
32     if((leds != leds_expected) || (out != out_expected)) begin
33         error_count++;
34         $display("Expected Leds:%b, Out:%d| DUT Leds:%b, Out:%d <---- ERROR", leds_expected, out_expected, leds, out);
35         //$stop;
36     end
37     else begin
38         $display("Expected Leds:%b, Out:%d| DUT Leds:%b, Out:%d", leds_expected, out_expected, leds, out);
39         correct_count++;
40     end
41 endtask
42 endmodule

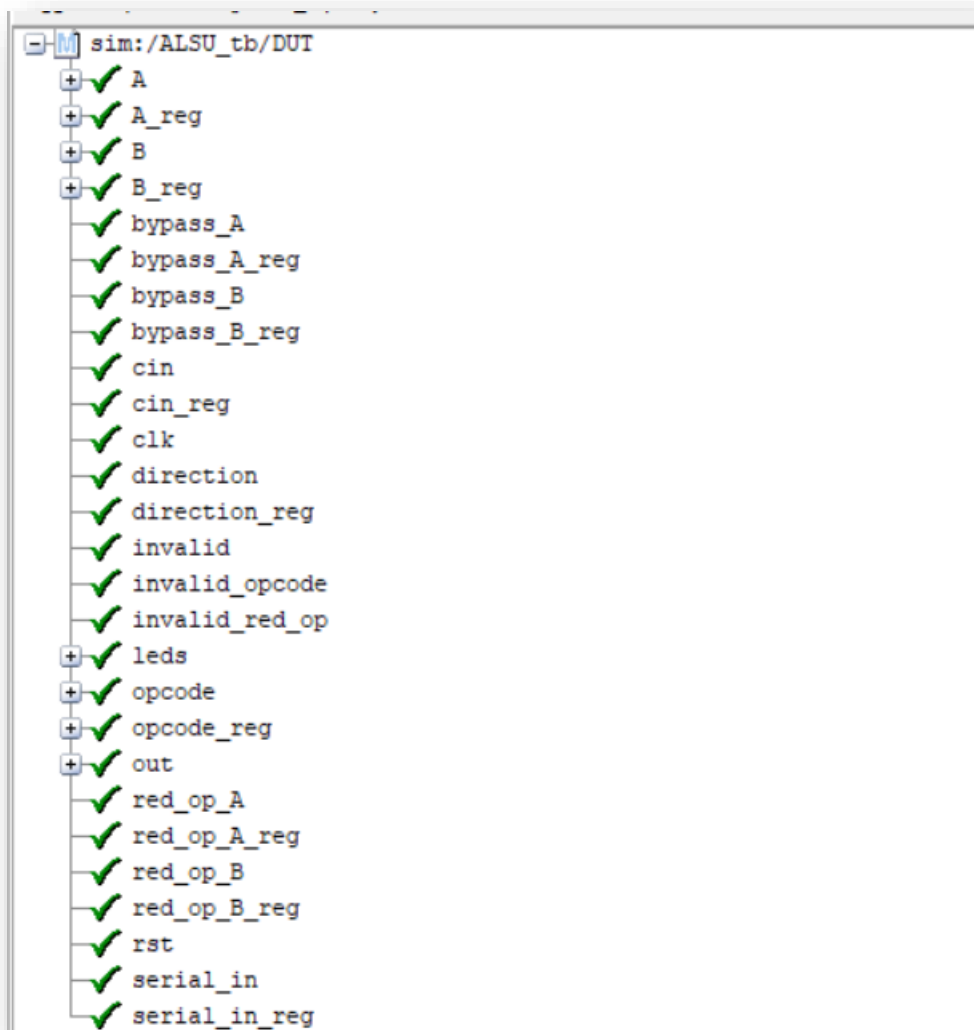
```

5. QuestaSim



6. Code Coverage:

Toggle Coverage:



Statement Coverage:

```
ALSU.v
18 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
19 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20 assign invalid = invalid_red_op | invalid_opcode;
23 always @(posedge clk or posedge rst) begin
25   cin_reg <= 0;
26   red_op_B_reg <= 0;
27   red_op_A_reg <= 0;
28   bypass_B_reg <= 0;
29   bypass_A_reg <= 0;
30   direction_reg <= 0;
31   serial_in_reg <= 0;
32   opcode_reg <= 0;
33   A_reg <= 0;
34   B_reg <= 0;
36   cin_reg <= cin;
37   red_op_B_reg <= red_op_B;
38   red_op_A_reg <= red_op_A;
39   bypass_B_reg <= bypass_B;
40   bypass_A_reg <= bypass_A;
41   direction_reg <= direction;
42   serial_in_reg <= serial_in;
43   opcode_reg <= opcode;
44   A_reg <= A;
45   B_reg <= B;
50   always @(posedge clk or posedge rst) begin
52     leds <= 0;
55     leds <= ~leds;
57     leds <= 0;
62   always @(posedge clk or posedge rst) begin
64     out <= 0;
68     out <= (INPUT_PRIORITY == "A") ? A_reg : B_reg;
70     out <= A_reg;
72     out <= B_reg;
74     out <= 0;
79     out <= (INPUT_PRIORITY == "A") ? |A_reg : |B_reg;
81     out <= |A_reg;
83     out <= |B_reg;

85   out <= A_reg | B_reg;
89   out <= (INPUT_PRIORITY == "A") ? ^A_reg : ^B_reg;
91   out <= ^A_reg;
93   out <= ^B_reg;
95   out <= A_reg ^ B_reg;
97   3'h2: out <= (FULL_ADDER == "ON") ? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
98   3'h3: out <= A_reg * B_reg;
101  out <= {out[4:0], serial_in_reg};
103  out <= {serial_in_reg, out[5:1]};
107  out <= {out[4:0], out[5]};
109  out <= {out[0], out[5:1]};
E   111 default: out <= 0;
```

Branch Coverage:

```
✓ 24 if(rst) begin
✓ 35 end else begin
✓ 51 if(rst) begin
✓ 53 end else begin
✓ 54 if (invalid)
✓ 56 else
✓ 63 if(rst) begin
✓ 66 else begin
✓ 67 if (bypass_A_reg && bypass_B_reg)
✓ 69 else if (bypass_A_reg)
✓ 71 else if (bypass_B_reg)
✓ 73 else if (invalid)
✓ 75 else begin
✓ 77 3'h0: begin
✓ 78 if (red_op_A_reg && red_op_B_reg)
✓ 80 else if (red_op_A_reg)
✓ 82 else if (red_op_B_reg)
✓ 84 else
✓ 87 3'h1: begin
✓ 88 if (red_op_A_reg && red_op_B_reg)
✓ 90 else if (red_op_A_reg)
✓ 92 else if (red_op_B_reg)
✓ 94 else
✓ 97 3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
✓ 98 3'h3: out <= A_reg * B_reg;
✓ 99 3'h4: begin
✓ 100 if (direction_reg)
✓ 102 else
✓ 105 3'h5: begin
✓ 106 if (direction_reg)
✓ 108 else
✓ 111 default: out <= 0;
```

7. Coverage Report

Branch Coverage:

```
=====
=== Instance: /\ALSU_tb#DUT
=== Design Unit: work.ALSU
=====
Branch Coverage:
  Enabled Coverage          Bins    Hits    Misses  Coverage
  -----
  Branches                 31      31        0   100.00%
=====Branch Details=====
```

Statement Coverage:

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	48	48	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\ALSU_tb#DUT --

Toggle and Total Coverage:

Toggle Coverage for instance /\ALSU_tb#DUT --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
A[0-2]	1	1	100.00
A_reg[2-0]	1	1	100.00
B[0-2]	1	1	100.00
B_reg[2-0]	1	1	100.00
bypass_A	1	1	100.00
bypass_A_reg	1	1	100.00
bypass_B	1	1	100.00
bypass_B_reg	1	1	100.00
cin	1	1	100.00
cin_reg	1	1	100.00
clk	1	1	100.00
direction	1	1	100.00
direction_reg	1	1	100.00
invalid	1	1	100.00
invalid_opcode	1	1	100.00
invalid_red_op	1	1	100.00
leds[15-0]	1	1	100.00
opcode[0-2]	1	1	100.00
opcode_reg[2-0]	1	1	100.00
out[5-0]	1	1	100.00
red_op_A	1	1	100.00
red_op_A_reg	1	1	100.00
red_op_B	1	1	100.00
red_op_B_reg	1	1	100.00
rst	1	1	100.00
serial_in	1	1	100.00
serial_in_reg	1	1	100.00

Total Node Count = 59

Toggled Node Count = 59

Untoggled Node Count = 0

Toggle Coverage = 100.00% (118 of 118 bins)

Total Coverage By Instance (filtered view): 100.00%

8. Functional Coverage

```

% Hit: 85.71% 100 -
Covergroup instance \\\ALSU_pkg::ALSU_inputs::CovCode
covered/total bins: 95.23% 100 - Uncovered
missing/total bins: 14 15 -
% Hit: 93.33% 100 -
Coverpoint A_CP 100.00% 100 - Covered
covered/total bins: 4 4 -
missing/total bins: 0 4 -
% Hit: 100.00% 100 -
bin A_data_0 126 1 - Covered
bin A_data_max 152 1 - Covered
bin A_data_walkingones[1] 4 1 - Covered
bin A_data_walkingones[2] 2 1 - Covered
default bin A_data_default 132 - Occurred
Coverpoint B_CP 100.00% 100 - Covered
covered/total bins: 4 4 -
missing/total bins: 0 4 -
% Hit: 100.00% 100 -
bin B_data_0 118 1 - Covered
bin B_data_max 124 1 - Covered
bin B_data_walkingones[1] 2 1 - Covered
bin B_data_walkingones[2] 4 1 - Covered
default bin B_data_default 134 - Occurred
Coverpoint ALU_cp 85.71% 100 - Uncovered
covered/total bins: 6 7 -
missing/total bins: 1 7 -
% Hit: 85.71% 100 -
illegal_bin Bins_invalid 30 - Occurred
bin Bins_shift[SHIFT] 76 1 - Covered
bin Bins_shift[ROTATE] 74 1 - Covered
bin Bins_arith[ADD] 90 1 - Covered
bin Bins_arith[MULT] 78 1 - Covered
bin Bins_bitwise[OR] 6 1 - Covered
bin Bins_bitwise[XOR] 70 1 - Covered
bin Bins_trans 0 1 - ZERO

TOTAL COVERGROUP COVERAGE: 95.23% COVERGROUP TYPES: 1

Total Coverage By Instance (filtered view): 95.23%

```

Note: Couldn't have Bins_trans to be covered even with directed testing

Q4)

1. Design:

```
1  module my_mem(  
2  input clk,  
3  input write,  
4  input read,  
5  input [7:0] data_in,  
6  input [15:0] address,  
7  output reg [7:0] data_out  
8  
9  );  
10 // Declare a 9-bit associative array using the logic data type & the key of int datatype  
11 logic [8:0] mem_array[int];  
12  
13 always @(posedge clk) begin  
14     if (write)  
15         mem_array[address] = {~^data_in, data_in};  
16     else if (read)  
17         data_out = mem_array[address];  
18 end  
19 endmodule
```

2. Test Bench:

```
1
2 module my_mem_tb();
3
4     parameter TESTS = 100;
5     logic [15:0] address_array[];
6     logic [7:0] data_to_write_array[];
7
8     logic [7:0] data_read_expect_assoc[int];
9     logic [7:0] data_read_queue[$];
10    logic [7:0] data;
11
12    //MEM INPUT and OUTPUT ports
13    logic clk;
14    logic write;
15    logic read;
16    logic [7:0] data_in;
17    logic [15:0] address;
18    logic [7:0] data_out;
19
20    int error_count, correct_count;
21
22    my_mem dut (
23        .clk      (clk),
24        .write     (write),
25        .read      (read),
26        .data_in   (data_in),
27        .address   (address),
28        .data_out  (data_out)
29    );
30
31    initial begin
32        clk = 0;
33        forever
34            #1 clk = ~clk;
35    end
36
37    initial begin
38        error_count = 0; correct_count = 0;
39        write = 0; read = 0; address = 0; data_in = 0;
40        stimulus_gen();
41        golden_model();
42
43        for(int i = 0; i < TESTS; i++)begin
44
45            write = 1; read = 0;
46            address = address_array[i];
47            data_in = data_to_write_array[i];
48            @(negedge clk);
49
50
51        end
52
53        for(int i = 0; i < TESTS; i++)begin
54            read = 1; write = 0; address = address_array[i];
55            @(negedge clk);
56            check9Bits(address_array[i]);
57            data_read_queue.push_front(data_out);
58
59        end
60    end
```

```

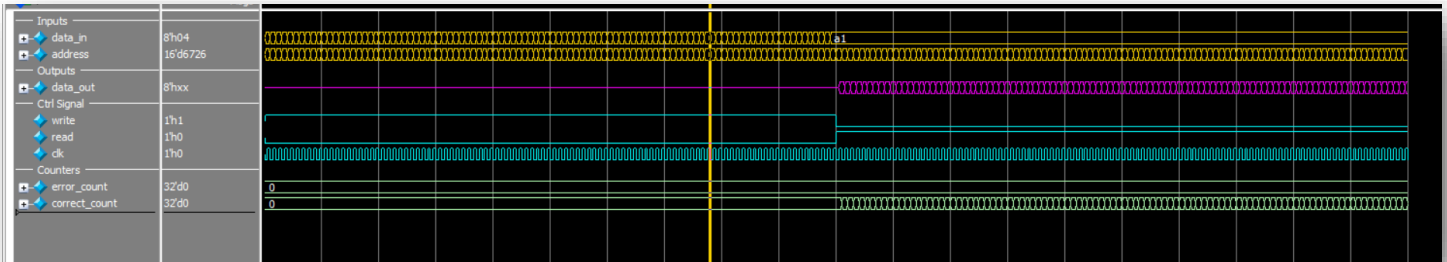
1
2     for(int i = 0; data_read_queue.size(); i++)begin
3         data = data_read_queue.pop_front();
4         $display("-Data Read:%h", i, data);
5     end
6
7     $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
8     $stop;
9 end
10
11 task stimulus_gen();
12
13     address_array = new[TESTS];
14     data_to_write_array = new[TESTS];
15
16     for(int i = 0; i < TESTS; i++)begin
17         address_array[i] = $random;
18         data_to_write_array[i] = $random;
19     end
20 endtask
21
22 task golden_model();
23
24     for(int i = 0; i < TESTS; i++)begin
25         data_read_expect_assoc[address_array[i]] = data_to_write_array[i];
26     end
27 endtask
28
29 task check9Bits(input logic [15:0] address);
30
31     if(data_out != data_read_expect_assoc[address])begin
32         error_count++;
33         $display("Error in Data Out: Expcted:%h | DUT:%h ", data_read_expect_assoc[address], data_out);
34         $stop;
35     end
36     else
37         correct_count++;
38
39 endtask
40
41 endmodule

```

3. Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
MEM_1	Call the stimulus_gen and then the golden_model tasks to prepare the data to be sent and expected data to be read from the RAM	Directed at the beginning to prep data	A checker to make sure output is correct
MEM_2	Loops TESTS time to test the write operations on DUT	Directed during stimulus generation after MEM_1	A checker to make sure output is correct
MEM_3	Loops TESTS time to test the read operations on DUT , and self-checks	Directed during stimulus generation after MEM_2	A checker to make sure output is correct

5. Questasim Waveform:



6. Questasim Output:

```
# 65.Data Read:15
# 66.Data Read:3a
# 67.Data Read:4f
# 68.Data Read:71
# 69.Data Read:2a
# 70.Data Read:ae
# 71.Data Read:b6
# 72.Data Read:89
# 73.Data Read:d8
# 74.Data Read:8a
# 75.Data Read:3c
# 76.Data Read:0a
# 77.Data Read:cf
# 78.Data Read:ae
# 79.Data Read:d5
# 80.Data Read:53
# 81.Data Read:13
# 82.Data Read:9d
# 83.Data Read:20
# 84.Data Read:0a
# 85.Data Read:65
# 86.Data Read:bd
# 87.Data Read:c5
# 88.Data Read:ce
# 89.Data Read:8f
# 90.Data Read:77
# 91.Data Read:aa
# 92.Data Read:c6
# 93.Data Read:8c
# 94.Data Read:3d
# 95.Data Read:0d
# 96.Data Read:12
# 97.Data Read:8d
# 98.Data Read:63
# 99.Data Read:81
# Total Tests:      100 | Correct Tests:      100 | Error Test:      0
```

7. Do File

```
1  vlib work
2  vlog memory_sv memory_tb_sv +cover -covercells
3  vsim -voptargs=+acc work.my_mem_tb -cover
4  do wave.do
5  coverage save memory_tb.ucdb -onexit -du work.my_mem
6  run -all
7  #quit -sim
8  #vcover report memory_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```